

Testing Strategy and Plan: Internship Progress Dashboard

This document outlines the comprehensive testing strategy and detailed test plan for the **Internship Progress Dashboard**, a full-stack web application designed for students and mentors to manage and track internship progress. The strategy focuses on quality, security, and performance across the entire technology stack.

1. Testing Types

A multi-layered testing approach will be adopted to ensure high product quality.

Unit Testing (Frontend + Backend)

Goal: To verify that the smallest testable parts (functions, components, classes) of the application work correctly in isolation.

Component	Example	Tool
Frontend (React.js)	Testing if the Login component renders correctly and handles button clicks.	Jest, React Testing Library
Backend (FastAPI)	Testing a function that calculates a student's progress percentage based on task data.	Pytest

Integration Testing

Goal: To verify that different modules or services used by the application (e.g., frontend-backend communication, database connection) work together as expected.

Example: Testing the flow where the Frontend sends a login request to the Backend, and the Backend successfully authenticates the user against the MongoDB database.

End-to-End Testing (E2E)

Goal: To simulate real user scenarios from start to finish, ensuring the entire application flow is functional.

Example: A complete user journey: A Student logs in, navigates to the dashboard, updates a task status, and the change is reflected in the database and the dashboard UI.

API Testing

Goal: To test the backend FastAPI endpoints directly by sending requests and validating the responses (status code, payload, headers). This is crucial for verifying business logic independent of the UI.

Example: Direct **POST** request to the **/tasks/create** endpoint with valid JSON data, expecting a **201 Created** status code and the created task object in the response body.

UI Testing

Goal: To verify the visual and interactive elements of the user interface.

Example: Checking that the Progress Dashboard displays the correct data layout on different screen sizes (responsiveness), and that interactive elements like filters and sort buttons work.

Security Testing

Goal: To identify vulnerabilities in the application's authentication, authorization, and data handling.

Example: Attempting to access an Admin-only endpoint (e.g., **/admin/assign_task**) using a Student's valid JWT token, expecting a **403 Forbidden** response.

Performance Testing

Goal: To determine the speed, responsiveness, and stability of the application under various load conditions.

Example: Simulating 500 concurrent students logging in and checking their dashboards to measure the application's maximum capacity and response time under heavy load.

Usability Testing

Goal: To assess how easy and intuitive the application is for the target users (Students and Admin/Mentors).

Example: Asking a small group of mentors to navigate the Task Assignment process without prior instruction and observing if they can successfully complete the task in under 60 seconds.

2. Test Cases (Structured)

The following tables provide detailed test cases for core features.

User Registration & Login Test Cases

Test Case ID	Description	Steps	Expected Result	Actual Result
AUTH-001	Successful User Registration (Student)	1. Navigate to the Signup page. 2. Enter valid email, password, and select 'Student' role. 3. Click 'Register'.	User is successfully created, redirected to the Login page, and a success notification is shown.	
AUTH-002	Login with valid credentials	1. Navigate to the Login page. 2. Enter valid email and password. 3. Click 'Login'.	User is successfully logged in, a valid JWT token is received, and the user is redirected to their respective Dashboard.	
AUTH-003	Invalid Password Error	1. Navigate to the Login page. 2. Enter a valid email. 3. Enter an incorrect password. 4. Click 'Login'.	A clear error message is displayed: "Invalid email or password."	

Task Management Test Cases

Test Case ID	Description	Steps	Expected Result	Actual Result
TASK-001	Successful Task Creation (Admin/Mentor)	1. Admin logs in. 2. Navigates to the 'Task Assignment' page. 3. Enters task title, description, and assigns it to a student. 4. Clicks 'Assign'.	The task is successfully created in the database and appears in the assigned student's task list. Status code 201 is returned.	
TASK-002	Task Status Update by Student	1. Student logs in. 2. Navigates to their task list. 3. Clicks on a task and changes the status from 'To Do' to 'In Progress'.	The task status is updated in the database. The dashboard reflects the change instantly (real-time update verification).	

Test Case ID	Description	Steps	Expected Result	Actual Result
TASK-003	Dashboard Data Accuracy	1. Student has 3 tasks: 1 'Done', 2 'In Progress'. 2. Student logs in and views the dashboard.	The dashboard displays "33.3% Complete" and counts are accurate (1 Done, 2 In Progress).	

JWT Authentication & Access Control Test Cases

Test Case ID	Description	Steps	Expected Result	Actual Result
SEC-001	Access Admin page with Student Token	1. Student logs in and obtains JWT token. 2. Attempts to make an API call to the Admin endpoint (/tasks/create).	The API returns 403 Forbidden status code. The request is denied.	
SEC-002	JWT Token Expiry	1. User logs in. 2. Waits for the configured token expiry time (e.g., 15 minutes). 3. Attempts to refresh the dashboard or make a subsequent API call.	The API returns 401 Unauthorized. The user is prompted to log in again.	
SEC-003	Invalid Input Error Handling	1. Attempts to register with an email in an invalid format (e.g., "test@.com"). 2. Submits the form.	The Frontend displays a validation error message: "Please enter a valid email address." No API request is sent.	

3. API Testing

API testing is essential due to the FastAPI/MongoDB backend. We will use a tool like Postman or Pytest for comprehensive coverage.

Scenario	Endpoint	Method	Authentication Header	Request Body (Example)	Expected Status	Response Validation
Register User	/auth/register	POST	None	{ "email": "...", "password": "...", "role": "student" }	201	Verify a success message and token (if immediately logged in).
Create Task	/tasks/create	POST	Valid Admin JWT	{ "title": "...", "assigned_to_id": "..." }	201	Check if the response contains the new task ID and details.

Scenario	Endpoint	Method	Authentication Header	Request Body (Example)	Expected Status	Response Validation
Get User Tasks	/tasks/my-tasks	GET	Valid Student JWT	None	200	Verify the response array structure and ensure only tasks assigned to the user are returned.
Unauthorized Access	/tasks/create	POST	Valid Student JWT	(Any valid body)	403	Verify the response body contains an "Unauthorized role" message.
Invalid Request Data	/auth/register	POST	None	{ "email": "...", "password": "" } (Missing password)	422	Check for a detailed FastAPI Pydantic validation error message.

4. Automation Strategy

Automated testing is crucial for continuous delivery (CI/CD) and regression prevention.

Tools and Frameworks

Layer	Tool	Purpose
Frontend	Jest & React Testing Library	Unit testing React components and custom hooks, focusing on component behavior and user interaction logic.
Backend	Pytest	Unit and Integration testing for FastAPI handlers, services, and database interactions (using test fixtures).
E2E	Cypress or Playwright	Simulating user flows across the entire application in a browser environment. This covers UI and backend integration.
CI/CD	GitHub Actions	Automated workflow for building, testing, and deploying the application.

Automated Testing in the CI/CD Pipeline

1. **Code Commit:** A developer pushes code to the `develop` or feature branch.
2. **Trigger:** GitHub Actions pipeline is automatically triggered.
3. **Build Stage (Docker):** The Docker image for the Frontend and Backend services is built.
4. **Backend Test Stage:**
 - Pytest runs all backend Unit and Integration tests.
 - **Action:** If tests fail, the pipeline stops, and feedback is provided.
5. **Frontend Test Stage:**
 - Jest/React Testing Library runs all frontend Unit tests.
 - **Action:** If tests fail, the pipeline stops.
6. **E2E Test Stage:**
 - Cypress/Playwright tests run against the staging deployment (or containerized services).
 - **Action:** If E2E tests fail, the pipeline stops.
7. **Deployment:** If all tests pass, the application is deployed to the staging environment.

5. Security Testing

A focus on security is necessary given the use of JWT and role-based access control (RBAC).

Security Area	Testing Goal	Example Test Scenario
JWT Token Tampering	Ensure the application rejects tokens that have been modified post-issuance.	Modify the payload of a valid token (e.g., change <code>role: "student"</code> to <code>role: "admin"</code>) and attempt to access an admin resource. Expected result: <code>401 Unauthorized</code> or token signature verification failure.
Unauthorized Access	Verify strict RBAC implementation.	Attempt to make API calls to endpoints restricted to other roles or users (e.g., Student tries to update another student's task).
NoSQL Injection Attacks	Test input fields that might directly interact with MongoDB queries.	Enter MongoDB-specific operators (e.g., <code>{"\$ne": 1}</code>) into login or search forms to try

Security Area	Testing Goal	Example Test Scenario
Input Validation	Confirm all inputs are sanitized and validated on the backend.	and bypass authentication or retrieve unauthorized data.
		Submit excessively long strings, negative numbers, or HTML/JS scripts (XSS attempts) to ensure the backend sanitizes or rejects them with a 422 Unprocessable Entity.

6. Performance Testing

We will use dedicated tools to ensure the application remains fast and stable under expected and maximum load.

Test Type	Objective	Metric	Suggested Tool
Load Testing	Determine application behavior under expected high user traffic.	Response Time, Latency, Throughput (Requests per second)	Locust
Stress Testing	Find the breaking point of the system by pushing it beyond its limits.	Error Rate, Maximum concurrent users before failure	JMeter
Response Time Validation	Ensure critical API endpoints (e.g., /tasks/my-tasks, /auth/login) respond quickly.	Average response time for critical endpoints (Target: < 500ms).	Locust

7. Bug Reporting Template

A standardized template ensures clear, actionable, and repeatable bug reports.

Professional Bug Report Format

Field	Description/Guidance
Title	A concise summary of the bug (e.g., "Admin task assignment button is unresponsive on mobile").
Severity	Critical: Production down, core feature blocked. Major: Significant feature defect, clear workaround exists. Minor: Cosmetic issue, non-blocking typo.
Steps to Reproduce	Clear, numbered steps to replicate the issue. (Example: 1. Log in as Student. 2. Navigate to Task view. 3. Click 'Filter by Status' dropdown.)
Expected Result	What should have happened if the feature worked correctly.
Actual Result	What actually happened (the erroneous behavior).
Environment	OS, Browser, Application Version, Role (e.g., Windows 11, Chrome v120, v1.0-staging, Admin).
Screenshot/Recording	Visual proof of the bug.
Attachment	 File Link to screenshot or screen recording of the bug